Scale-Invariant Baselines Can Certify a Broken Deep Model: Grouped-Normalisation Collapse in Learned Trajectory Screening

Abstract

A single feature scaler is often used among a set of data. We show that it can quietly harm a sequence model without warning the practitioner about any problems, even when they apply all the checks they should. A Transformer trained with a scaler averaged over a target group ceases to react to the input on Venus (3.11e-05 prediction standard deviation on unseen missions, 0.6037 validation area under the ROC curve (AUC)). Let's train a gradient boosted tree on the same arrays and nothing is amiss. Its AUC varies by at most 0.0002 across all three normalisation settings, and it finds the task almost perfectly separable while the network cannot separate it at all. Trees here are not affected by the thing that is destroying the network. Severe, but not everywhere - 1 of 4 targets collapses; the remaining ones lose between 0.0003 and 0.0925 AUC while they keep working. The compression is the same across the 5 seeds, and the collapse occurs on 4 of them, it is not due to one unlucky initialisation but is inherent in the preprocessing itself. Our AI contribution is a controlled test of when pooled normalisation breaks a sequence model (survivable across timesteps, not across groups), and a check on prediction spread that catches it, with its threshold set on validation runs and tested on held out ones. On NASA turbofan data the signal also shrinks about nine times, but the model keeps working. The engineering application is screening simulated spacecraft trajectories to save compute.

Keywords: feature normalisation, shortcut learning, model diagnostics, simulation screening, trajectory analysis, negative results

1. Introduction

In Monte Carlo trajectory campaigns, the majority of the compute is used on runs that were already sure to fail when they left the parking orbit. These campaigns are a regular occurrence at flight facilities, such as the

large dispersion campaign that TESS used to set its trajectory design (Nickel et al., 2016). Not only is this expensive, a body of work has been developed to make each propagation even less expensive (Massari et al., 2017; Jia and Xin, 2022). Another, and obvious, approach is to steer clear of these runs: track the first part of a trajectory and cancel out the ones that will fail. It's a screen we have made. It revealed a failure that we believe is much more than astrodynamics related.

This paper deals with that failure. Grouped data was used for training the model and a single feature scaler was fitted for the entire group. It ended up emitting a single constant per group. It's not the surprising part that it happened. What's most interesting is the length of time it took to be noticed. With a mixed validation set it looked fine, since a model can score well if it differentiates groups but cannot differentiate anything within the groups, and validation AUC remained high. A baseline run was executed with a gradient boosted model, with almost perfect results, because the trees divide on the actual values and the scaling on which the network depends is not important to them. Both of the tests we would have used would have told us that the pipeline was healthy.

This isn't, of course, comfortable, as running a simple baseline is good advice. It is common practice for tabular and physical data to be checked by tree ensembles and they are typically the correct choice (Chen and Guestrin, 2016; Grinsztajn et al., 2022; Shwartz-Ziv and Armon, 2022). In fact, the very property that makes them good checks in general (invariance to the scale of the features) is the one that makes them insensitive to this specific defect. The check cannot fail the model, so it passes it.

Someone could say that this is only a bad scaling choice. It is, and the fix is easy. The fix is not what we are writing about. We are writing about how this failure hides. The usual validation does not show it, and the usual baseline cannot show it. We describe the failure, show why those two checks miss it, and give a check that does not miss it.

We make three contributions. It is first a controlled decomposition of the collapse, which distinguishes pooling over time from pooling across groups, and shows only the latter to be deadly. Second, a demonstration that a scale-invariant baseline actually approves of the broken configuration, as well as an inexpensive diagnostic, the spread of the predictions, that does notice it. Its threshold is set on validation runs and tested on runs it has not seen. Third, a second dataset, NASA turbofan engines, where the signal shrinks in the same way and the model keeps working, which shows that the shrinking

alone is not the problem.

This paper is based on scripts in the attached repository that generate all the numbers, which are written to machine-readable artefacts. Those artefacts are then used to create the paper containing tables and figures.

2. Related Work

Normalisation is crucial for training deep networks, and there are standard layers (batch, layer, group, instance) that solve a version of the problem (Ioffe and Szegedy, 2015; Ba et al., 2016; Wu and He, 2018; Ulyanov et al., 2016; Huang et al., 2023). It has been long known that the ability of a network to train, and not merely the rate at which it trains, depends on the scaling of the inputs (Sola and Sevilla, 1997). These layers also make some assumptions about the input that they're given. They assume that the feature statistics are stationary, or that they're comparable within each pooling axis. When more than one set of data is measured on the same scaler, but the range of the data is very different, that assumption is already broken before the first layer runs. Normalisation has also been studied for time series on its own. ST-Norm splits it into a temporal part and a spatial part for forecasting models (Deng et al., 2021), and Deep Adaptive Input Normalization learns the scaling from the data (Passalis et al., 2020). Those papers want a more accurate model. We ask a different question: how can a bad scaling choice break a model without anyone noticing? What we add is a controlled decomposition that shows us that it is not the pooling in general that does the damage, but only the pooling across the groups, and then an observation that the resulting model is not a weak model, but a constant model.

When a shortcut is there, a network will easily learn the easy signals and take the shortcut over the intended structure (Geirhos et al., 2020; Shah et al., 2020). Previous research on Clever-Hans behaviour demonstrates that the models achieve good scores for reasons that are not related to the task (Lapuschkin et al., 2019). If the scales are grouped and are not homogenous, there is an exceptionally clean shortcut, the distance between the group centroids. This type of model simply memorizes the difference, and has validation AUC of >0.95 but performs no better inside individual groups. That's what we saw. Here, our contribution is a diagnostic (spread of the predictions within a group) which does not need any label and is quite different from the common underfitting. It is also not a modelling problem, but a pipeline level

problem, like many which can be found in the shared preprocessing code that no single model owns (Chaudhary et al., 2018).

Tree ensembles perform well on tabular and physical data, and often perform with the same or better results than deep models (Chen and Guestrin, 2016; Grinsztajn et al., 2022; Shwartz-Ziv and Armon, 2022; Ke et al., 2017; Pasaribu et al., 2024), and most practitioners turn to a tree as a baseline for that reason. Their splits are axis-aligned and threshold-based, with a monotone rescaling of a feature having no effect on the splits. It's typically a good thing that it's invariant. It also makes them unable to identify a deadly error in the preprocessing, an error that results in a failure of the network.

Networks have been successfully applied in astrodynamics for trajectory optimization, approximation of optimal control (Izzo et al., 2019a, 2024; Sánchez-Sánchez and Izzo, 2018), cost estimation without complete propagation (Li et al., 2020b; Izzo et al., 2019b) and learning of a robust low thrust policy with certification via Monte Carlo (Zavoli and Federici, 2021). Learned models also correct orbit predictions from sparse tracking (Li et al., 2020a), stand in for Monte Carlo when propagating orbit uncertainty forward (Zhou et al., 2024), and are used for anomaly and fault detection in spacecraft telemetry (Ibrahim et al., 2019; Naik et al., 2020), where the same family of attention architectures is used as here (Jiang et al., 2023). Screening specifically, by means of a learned model selecting which of a very large number of candidate cases get full analysis, is already recognized: an ESA competition on real conjunction data (Uriot et al., 2020; Acciarini et al., 2021), and a benchmark of deep approaches against the classical filters they would replace (Stevenson et al., 2023). The final one is the most similar setting, a screening task on a deterministic astrodynamics simulator, measured against non-deep baselines. We could not find an article in this literature that considers early termination of individual Monte Carlo trajectory runs as we do in Section 8. The closest work decreases the cost of each propagation rather than deciding which propagations to start (Massari et al., 2017; Jia and Xin, 2022). The general idea is to use a learned surrogate instead of part of an expensive simulation (Yin et al., 2022).

3. Setup

In total, we developed 70,000 missions that leave the Earth, 10,000 missions to each of seven interplanetary targets (Mercury, Venus, Mars, Jupiter, Saturn, Uranus and Neptune), using NASA's General Mission Analysis Tool.

It has an extensively verified two and three body propagator (Hughes et al., 2014), and is deterministic. Each mission has six injection offsets, which are the components of the trans orbit insertion burn (`dv_V`, `dv_N`, `dv_B`), and the RAAN, AOP and INC of the parking orbit from which the mission departs. Each is classified as success or failure, and failed ones are further sub-categorised as surface impact, orbit too high, missed target and a few others.

The interplanetary transfers are sampled at a fixed rate of 54,000 s (15 hours) and downsampled to about 100 steps for each mission. This is so that the sequences produced by targets with flight times that vary by 2 orders of magnitude are comparable. Thirteen features are recorded for each step, and include position in the synodic frame, specific orbital energy, flight path angle, normalised target distance and eccentricity. An eighth target was created (the Moon), and is intentionally left out of the study. It is a 6 day transfer inside Earth's neighbourhood, sampled at 60 s, and it doesn't have a cost structure similar to Section 8, or a heliocentric dynamical regime.

The screening task is to observe the first 40% of the mission's flight path and decide whether or not to abort. Splits are mission based 70/15/15, deterministic in (n, seed). The experiments are all based on a single definition in the repository, so no model is ever scored using data that was used for model selection. It is not a theoretical question. Two of the previous analyses in this project shuffled the same number of missions using the same seed, and in those analyses the same set of missions ended up in both 'test' sets.

The sequence model is a Pre-LN Transformer encoder, `d_model` 128, 8 heads, 4 layers and CLS token pooling, and 2 heads are connected to a single trunk (one for mission outcome and one for failure mode). It is trained on random length prefixes, so that its predictions are not only at the 40% mark in the stream but at any point in the stream. The baseline is XGBoost, 300 trees at depth 5, fitted to the same normalised prefix, flattened. It is important to note that the baseline is not working from a feature set derived separately, but from the same view the network sees, which is what 'the same input' means throughout this paper. That detail carries most of the argument and we're very rigid with it.

The three conditions are the following sets of normalisations, which only vary in how the feature statistics are combined. All use the same architecture, seed, split, optimiser and epoch budget.

per timestep: every one takes its statistics from the distribution of features at that timestep index, over the missions of one target (Fig. 1). This

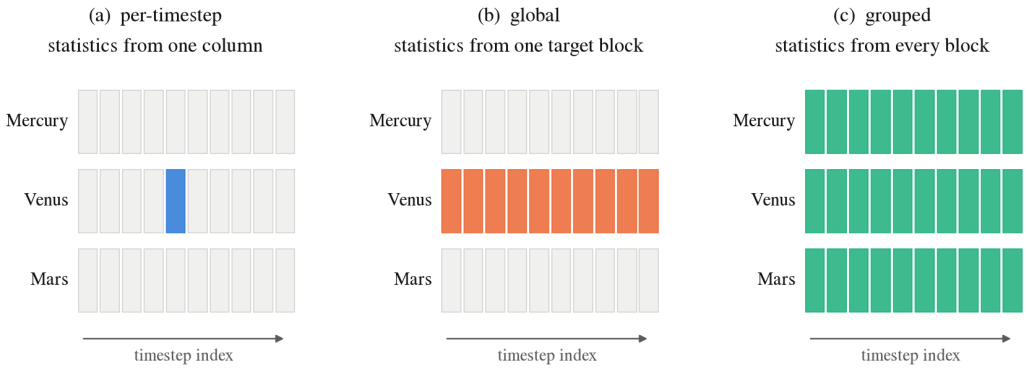


Figure 1: The three normalisation modes, and the only difference between them: from what part of the data does the scaler get its median and IQR? Everything else in training is the same. Each cell is all the missions of one target at one timestep index, and the shaded cells are the ones the median and IQR are estimated from.

is the configuration the working system uses.

global: one RobustScaler (median and IQR) per target, pooled across that target's own timesteps.

grouped: one RobustScaler pooled across all timesteps of all targets in a regime group (inner: Mercury, Venus, Mars; outer: Jupiter, Saturn, Uranus, Neptune). This is the setting that was not successful in production.

The third setting isn't a straw man, and here's why. You get a single RobustScaler from nearly all the standard pipeline tutorials, applied to a concatenated training set. No warning is issued and it looks like a normalised array.

4. Grouped-Normalisation Collapse

A mission's features sweep a wide range as they fly, and the range other targets occupy varies by orders of magnitude. A scaler applied to a set of pooled data will then scale the data on the scale of the largest source of variation in the set. What the screen should be able to tell apart is neither of those things. It is the spread between missions at a particular point in the flight, which by comparison is small. Pooling divides that spread by an arbitrary denominator, which drives the level of the useful signal towards zero and has no effect on the structure between groups. The network is then

tasked with learning a difference that is close to the limit of what gradient descent can detect, and Pre-LN layer normalisation filters out a lot of it.

This is used as the signal ratio, defined as the average of the standard deviation of the normalised features across missions, calculated only in the observed window. It's not a limitation you would expect but it does matter. The metric is totally useless averaged over the entire flight, because late timesteps have a huge spread across missions, as the missions that fail have physically diverged by that time, and that spread swamps the early signal so that all conditions appear healthy. The model commits at 40%, so what's important is the range within the prefix the model observes.

The first result is the decomposition that is controlled. The network is able to operate with the signal being reduced by up to a factor of 23 by pooling across time in a single target. It works on all 4 targets, with a loss of a few thousandths of AUC at most. Another kettle of fish is the pooling of a group. On Venus, the output standard deviation is 3.11e-05, which is one number for every mission, and the AUC drops from 0.9999 to 0.6037.

The term 'collapse' is used instead of 'underfitting' because of Fig. 2. A low AUC on its own doesn't mean much, because that is what you would get if the model learned something poor or wrong. A distribution on unseen missions that is three ten-thousandths wide is not ambiguous at all. The network is no longer under the control of its input.

The second result is that the collapse is selective, and we report that alongside the case that breaks. In the remaining 3 targets the same manipulation leaves the output still varying, and costs between 0.0003 and 0.0925 AUC. The signal ratio is not a measure of severity either. Jupiter compresses its signal to 0.0247, whereas Mercury compresses its signal to 0.0484, and yet Jupiter loses 0.0003 AUC whilst Mercury loses 0.0925 AUC. This is because Jupiter's task is nicely split up so that even a very weak signal is sufficient. Compression is necessary for the collapse but not sufficient; how hard the underlying task is modulates it. We are not claiming that these data support a rule of the form 'it collapses when the ratio is below X'. The counter example is visible in Fig. 3.

One note on scope. This ablation isolates normalisation and no other component of the model, so only the statistics are pooled, with one model trained for each target. The production failure also shared a single trunk across the group, which compounds the problem, since one trunk then has to serve targets whose inputs have all been flattened in the same direction, towards a constant. These numbers therefore represent a minimum damage

level of the configuration. They show that pooled normalisation is sufficient on its own to destroy a target, and not that this is a complete reconstruction of the original incident.

Under the grouped setting the tree reports AUC 1.0000 on Venus, the target where the network has collapsed to a constant at AUC 0.6037. Tree AUC changes by at most 0.0002 across settings on all targets, which implies that the choice is not really visible to the tree. Now if one runs it and checks whether or not the features have signal, doing the right and proper thing, he will get an unambiguous yes, and reasonably assume that the deep model needs tuning, and not that the pipeline is broken.

This is the methodological part and it's crucial to pay attention to the exact claim. The information is there and a tree can use it. The baseline is not informative on the question that has to be answered, which is 'can the network use it?'. Invariance to feature scaling is typically a good reason to reach for a tree as a diagnostic, and is precisely why it is blind here. The check is orthogonal to the defect by construction, so no matter how carefully one works the check could not have helped.

Accuracy on a mixed validation set does not catch the collapse, because a per group constant ranks the groups correctly and a pooled metric rewards that. It's picked up by two checks.

Prediction spread within a group. When a model is collapsed there will be very little variation of the model's output for varying input. This is a property of the predictions alone, needs no labels, and is quite different from ordinary underfitting, where the predictions are wrong but still vary.

Tree against network under the same preprocessing. If the tree has a big lead with both fed the same array, then the information is there and the network can't use it, rather than the information being absent.

Both are inexpensive and neither necessitates that you think you are going to come across this particular defect beforehand, and that's the important thing that makes a diagnostic worth having. When building a model on grouped data with scales that differ, we would recommend reporting the prediction spread per group in addition to the overall numbers. In our case it would have turned a problem that survived a series of review cycles into a one line check.

Table 1: Normalisation ablation. Identical architecture, seed and split; only the pooling differs. 'sig' is the mean within timestep standard deviation of the normalised features in the observed window. 'std' is the standard deviation of P(fail) on unseen missions, the quantity that identifies a collapse, as against a model that is merely inaccurate.

	per-timestep			global			grouped		
Target	sig	AUC	std	sig	AUC	std	sig	AUC	std
Venus	0.7681	0.9999	4.72e-01	0.0518	0.9947	4.50e-01	0.0023	0.6037	3.11e-05
Mars	1.0829	0.9994	4.33e-01	0.0472	0.9972	4.00e-01	0.8005	0.9422	4.06e-01
Mercury	0.7686	0.9994	4.27e-01	0.1321	0.9970	4.23e-01	0.0484	0.9069	4.25e-01
Jupiter	0.7708	1.0000	4.78e-01	0.0763	1.0000	4.78e-01	0.0247	0.9997	4.75e-01

Table 2: XGBoost on the same normalised window, for each setting. The tree is not affected by the preprocessing which destroys the network.

Target	per-timestep	global	grouped	spread
Venus	1.0000	1.0000	1.0000	0.0000
Mars	0.9996	0.9998	0.9998	0.0002
Mercury	0.9997	0.9998	0.9998	0.0001
Jupiter	1.0000	1.0000	1.0000	0.0000

5. Does It Reproduce Across Seeds?

Is it spread by seed? All of the above is problematic in that it's one run deep. A bad initialisation could result in an AUC of only 0.6037. Thus, we redo the whole ablation in 5 different seeds, in each of which we redraw the train/validation/test partition and the initialisation.

Compression is not dependent on seed. The signal ratio for Venus under grouped normalisation lands between 0.00222 and 0.00232 across all 5 runs, a spread of 5% against the compression of 335x itself. The healthy condition is stable too. The difference in per timestep AUC between two different seeds is at most 0.0010. There is no seed on which Mars, Mercury or Jupiter collapse.

It depends on the seed whether the network escapes or not. Venus collapses to a constant on 4 of 5 seeds, with prediction spread on unseen missions ranging from 1.57e-05 to 4.84e-05, which is 4 orders of magnitude lower than the 4.66e-01 obtained by correct normalisation. On the remaining seed it doesn't collapse, and it doesn't recover either: prediction spread 3.64e-02, still an order of magnitude short, at AUC 0.9305 against a healthy 0.9999.

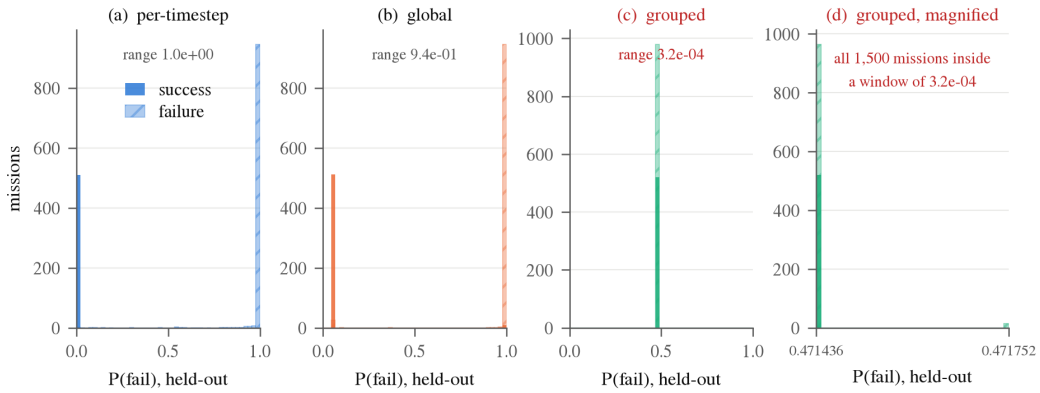


Figure 2: P(fail) on Venus, split out for each setting. Per timestep and global normalisation put the two populations at opposite ends of the axis. Under grouped normalisation every mission gets essentially the same number. Panel (d) magnifies that spike and shows the full range covers about three ten-thousandths. This is what makes a collapse different from a merely bad model.

The preprocessing is then always sufficient for the collapse, but the collapse itself is not certain, only the usual outcome.

This is the real reason for the prediction spread diagnostic above. The range of grouped AUC for Venus over these runs is 0.5338 to 0.9305, which overlaps that of a merely weakened target. Mercury sits inside that range on every seed while still telling missions apart. AUC is thus unable to distinguish the two states. Prediction spread can. There are orders of magnitude between the collapsed runs and all the healthy runs. If you have to keep tabs on one number for this failure, watch that one.

The blindness of the baseline is repeated to a greater degree. Across 5 seeds, 4 targets and three normalisation settings, which is 60 fitted trees in total, the tree's AUC stays within the range 0.9989 to 1.0000, and within each seed and target the AUC difference is at most 0.0003 between settings. There is no seed on which the baseline notices. That matters more than the number of collapses. The collapse is one target's misfortune, but a check that can't fail is a property of the check.

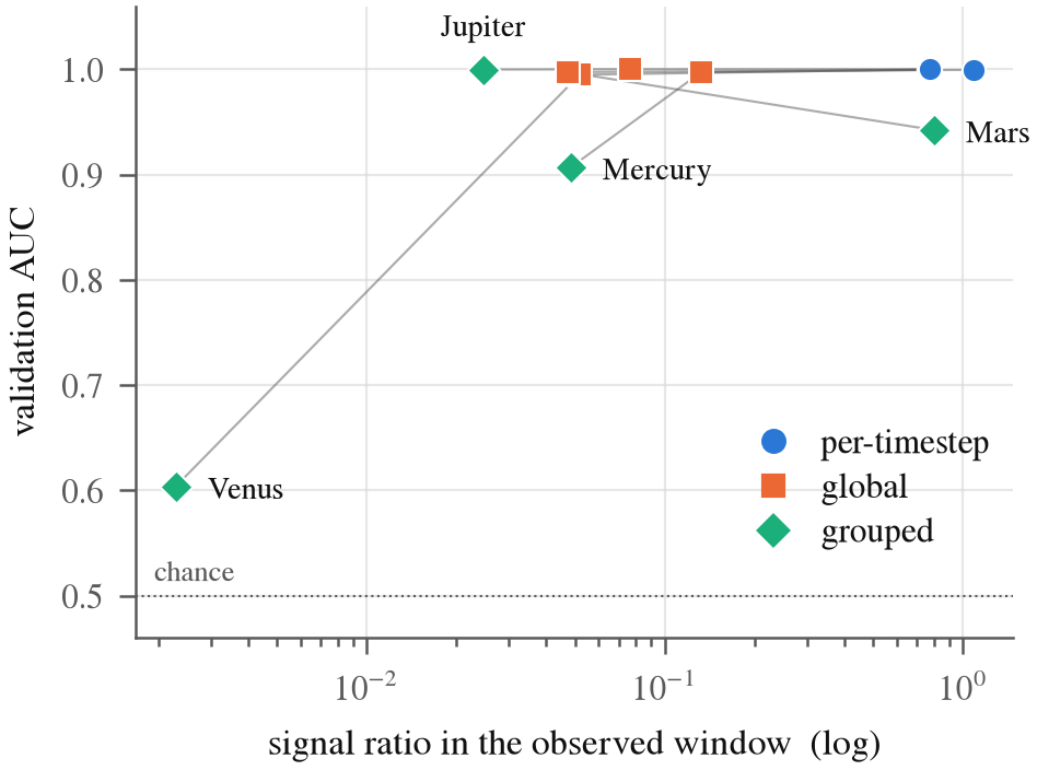


Figure 3: Signal ratio against validation AUC. These points would lie on a curve if the collapse was only due to compression. Jupiter has a lower signal ratio than Mercury and still loses very little.

6. A Check That Finds the Collapse

A collapsed model gives nearly the same P(fail) to every mission. So we look at how much its predictions vary. The score is

$$S = 2\,\mathrm{Std}(\hat{p}), \tag{1}$$

where $\hat{p}$ is the predicted P(fail) on the held out missions of one target. It is 0 for a constant output, and 1 for the widest spread a probability can have. It needs no labels and no second model.

The second score is the one a reviewer would try first. It divides by the spread of a healthy model on the same data:

$$D = \frac{\mathrm{Std}(\hat{p})}{\mathrm{Std}(\hat{p}_{\mathrm{healthy}}) + \epsilon}. \tag{2}$$

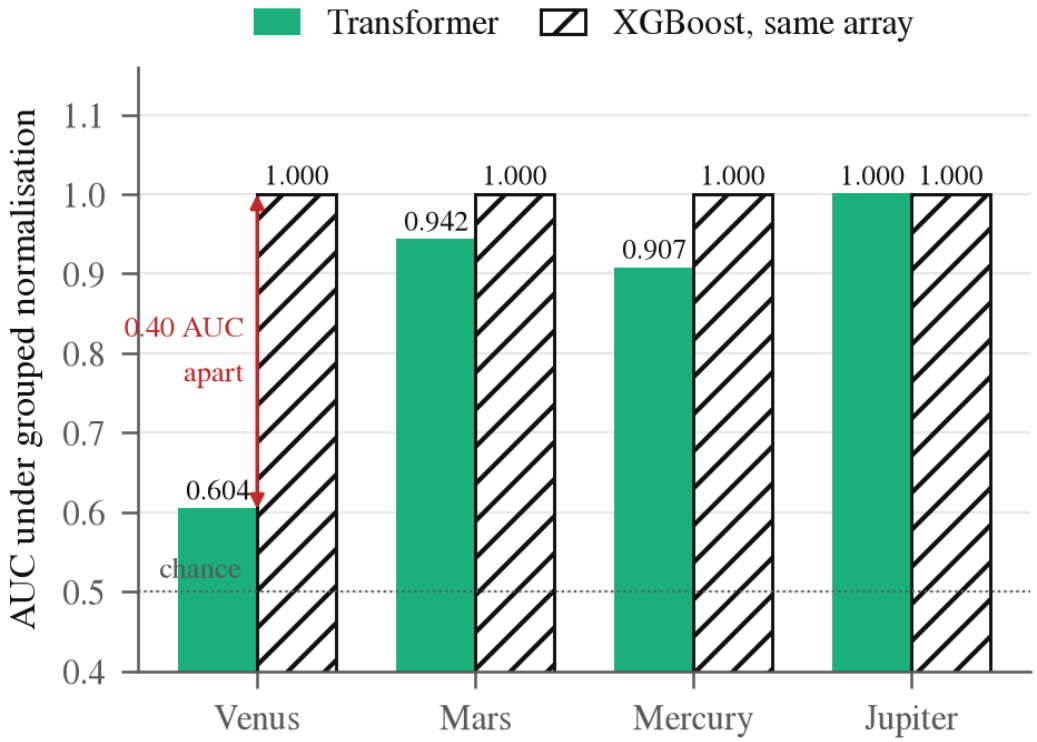


Figure 4: Both models under grouped normalisation, on the same arrays. The tree is at the ceiling on every target, including the one where the network is just above chance.

It needs a model that was trained the right way, and when something is broken that is the thing we do not have. So we use S as the main score and report D to compare.

To get models in every state we repeat the ablation of Section 4 on 5 seeds, 4 targets and 3 settings, which is 60 runs, and keep the predictions on the validation and the test missions of each run. The settings give four states. Per timestep runs are healthy. Global runs are mildly hurt. Grouped runs that keep working are severely hurt. Grouped runs that cannot rank missions are collapsed. A run counts as collapsed if its AUC on that split is 0.85 or lower. That cutoff sits in the biggest gap in the data: four runs fall between 0.51 and 0.82, and the rest are at 0.895 or above.

We chose the threshold on the validation predictions only. It is the middle, on a log scale, of the gap between the largest S of a collapsed run (1.0e-04) and the smallest S of a working run (0.071), which is 2.7e-03. Then we

Table 3: Validation AUC across 5 seeds, mean and standard deviation, and the number of seeds on which the grouped run collapsed to a constant output.

Target	per-timestep AUC	grouped AUC	collapsed
Venus	0.9998 ± 0.0001	0.6912 ± 0.1530	4/5
Mars	0.9992 ± 0.0004	0.9457 ± 0.0093	0/5
Mercury	0.9997 ± 0.0002	0.9118 ± 0.0036	0/5
Jupiter	0.9999 ± 0.0002	0.9997 ± 0.0003	0/5

Table 4: The 60 runs by state. S is the spread score on the test missions, and AUC is the median on the test missions. The spread of the collapsed runs is at least 750 times smaller than that of any other run.

State	Runs	S (smallest to largest)	Median AUC
Healthy	20	0.807 to 0.963	0.9996
Mild	20	0.748 to 0.962	0.9955
Severe	16	0.073 to 0.961	0.9430
Collapsed	4	3.15e-05 to 9.69e-05	0.5912

applied it, unchanged, to the test predictions. It flags 4 runs out of 60, and those are exactly the 4 that collapsed. No working run is flagged. D, with its threshold chosen the same way, flags the same 4. So the reference model did not add anything here.

The score is for collapse, not for every loss of accuracy. The Mercury runs under grouped normalisation lose about 0.09 AUC and their spread stays above 0.76, so the check does not flag them. Venus on seed 3 is the hard case. Its S is 0.073, which is about ten times below a healthy run, and it still ranks missions at an AUC of 0.92. It is above the threshold and is not flagged. If a run is called collapsed at AUC 0.90 and not 0.85, there are 5 collapsed runs and the ROC AUC of S is 0.993 and not 1.000.

The signal ratio of Section 4 can be measured before any training. Every collapsed run has it at 0.0022 or 0.0023. Five runs are below 0.01, which are the 4 collapsed runs and Venus on seed 3, and no run at 0.0243 or above collapsed. So it works as an early warning, but it cannot say which of those runs will collapse. For that, the spread after training is needed.

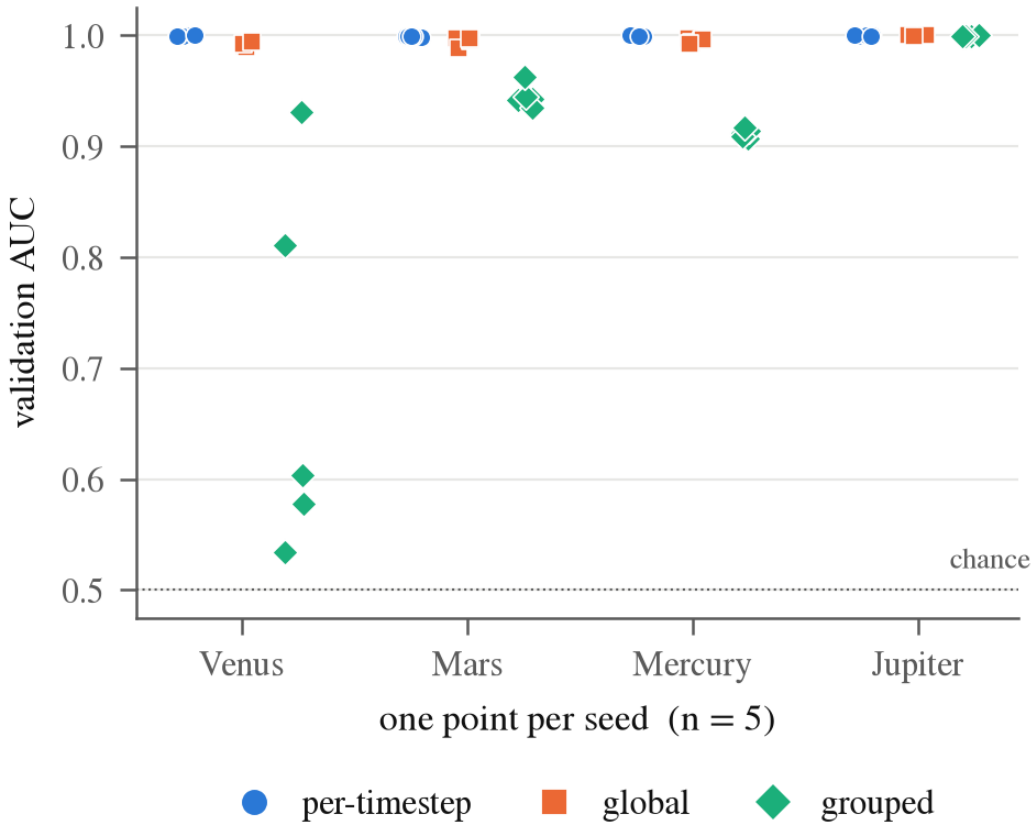


Figure 5: Every seed plotted on its own rather than summarised as an interval. With n = 5 the spread of the raw points is the honest summary.

7. A Second Dataset

To see what happens outside our simulator we used the NASA C-MAPSS turbofan data (Saxena et al., 2008), which is a standard public set for remaining useful life work (Li et al., 2018). It has run to failure sensor records for fleets of engines. FD001 has one operating condition. FD002 and FD004 have six. The six conditions play the role the targets play in our data: the sensors sit at very different levels in each one, and the slow drift that shows an engine is wearing out is small next to that gap.

The task is to look at the last 30 cycles of an engine and say whether it will fail within 30 cycles. The split is by engine, 70/15/15, so no engine is in two sets. We scale the 14 sensors that change over an engine's life in two ways. Pooled uses one median and IQR for every row. Per group uses

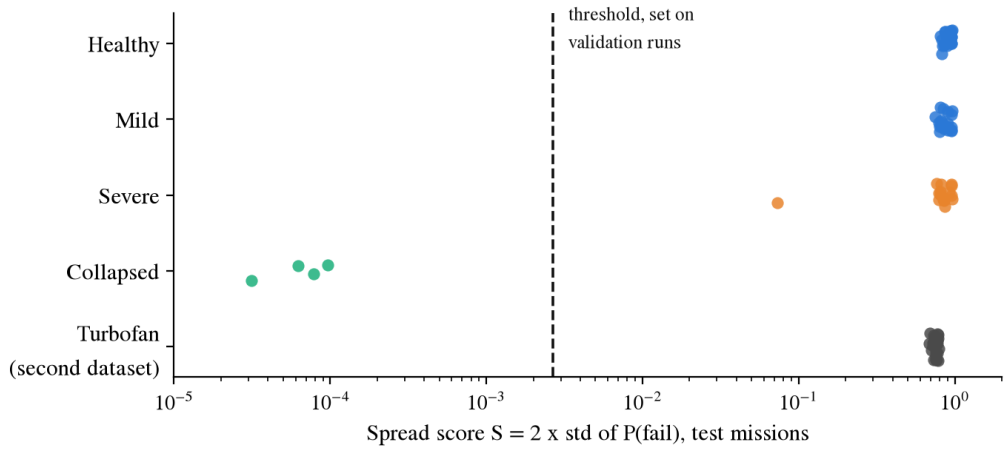


Figure 6: The spread score of every run on the test missions. The dashed line is the threshold chosen on the validation runs. The turbofan runs of Section 7 are shown for comparison, and none is flagged.

one for each operating condition. The model is the Transformer of Section 3, with the same optimiser, trained for 20 epochs on 5 seeds, and the tree is XGBoost on the same scaled window. FD001 has a single condition, so the two scalings are the same thing there. It is our control.

Pooling shrinks the signal about nine times, from 0.802 to 0.083 on FD002 and from 0.859 to 0.097 on FD004. The model does not break (Table 5). On both subsets the pooled Transformer is slightly ahead of the per group one, the spread of its predictions is the same, and the tree is fine either way. The check of Section 6, with the threshold it already had, flags none of the 30 runs.

This agrees with Section 4. The signal shrinking is not enough to break a model. In our data, Mercury and Jupiter lose most of their signal under grouped scaling and keep working. Here the drift is still there after pooling, and the model finds it. It also shows that the check does not raise an alarm on a second kind of data.

8. Where the Screen Belongs

If it has been thoroughly done on the sequence model, the real answer is that it should not be used. The six feature classifier, which is evaluated prior to any propagation whatsoever, saves 64.5% of compute against the telemetry

Table 5: Turbofan data, 5 seeds. 'signal' is the mean within group standard deviation of the scaled features. AUC is on test engines, mean and standard deviation. S is the spread score of the Transformer.

Subset	Scaling	signal	Transformer AUC	Tree AUC	S
FD001	pooled	0.784	0.9916 ± 0.0017	0.9953	0.760
FD001	per-group	0.784	0.9916 ± 0.0017	0.9953	0.760
FD002	pooled	0.083	0.9898 ± 0.0014	0.9862	0.767
FD002	per-group	0.802	0.9855 ± 0.0025	0.9925	0.755
FD004	pooled	0.097	0.9910 ± 0.0010	0.9890	0.734
FD004	per-group	0.859	0.9881 ± 0.0044	0.9933	0.731

Table 6: Screening economics. Compute is charged in propagation days, which span about a 100x range across the seven targets. Thresholds are fitted on validation and reported on the untouched test split, so the recall column is measured out of sample rather than achieved by construction.

Screen	Compute saved	Good missions lost	Failure recall
T0: six launch parameters, before propagating	64.5%	0.76%	99.06%
T40: telemetry Transformer at 40%	38.3%	0.21%	98.87%
Cascade: T0 where confident, else T40	65.0%	1.62%	99.94%

model's 38.3%, at a similar cost in good missions destroyed. There is also a levelling off of accuracy between 10% and 40% observed. The problem is not a modelling fault. The simulator is deterministic, the outcome is a fixed function of the injection offsets, and the trajectory is an integral of the offsets. It can't transport information that the parameters don't have.

This is not to imply that the task is trivial. Logistic regression on the same 6 features achieves 0.4824 to 0.5410 AUC on all of the targets, which is chance, so the mapping between the parameters and the outcome is very non-linear and requires a learned model. A temporal one is not really necessary. The sequential screen is only worth applying if the outcome is not known at injection: mid flight stochasticity, unmodelled dynamics, sensor noise, or campaigns where the launch parameters are not known. That is exactly the setting reinforcement learning trajectory work needs when it certifies policies through dispersed Monte Carlo campaigns (Zavoli and Federici, 2021).

9. Limitations

- Seeds. Replication: 5 seeds. This is enough to exclude an unfortunate initialisation, but not to warrant a narrow confidence interval. We report the spread of the individual runs and not a fitted interval. The collapse itself comes at 4/5, and the exception is likely subject to a controlling factor, but that just isn't known with a small sample.

- Datasets. The collapse is shown on the GMAT corpus. On the turbofan data the signal shrinks in the same way and the model keeps working.

- Four targets in the ablation. 1 of 4 targets collapses. All four agree on the result concerning the blind baseline, which is the half concerning method.

- The check. The 4 collapsed runs in the 60 are all Venus, so the threshold has been tested on one kind of collapse. It does not flag a model that loses accuracy and keeps its spread, such as Mercury.

- Synthetic and deterministic. There are no execution errors, no unmodelled accelerations, no sensor noise and all missions of a target have a common time base. This common time base is what makes standardisation per timestep possible at all and it is not a property of a real campaign, but of the generator.

- Real mission data. Public satellite telemetry benchmarks exist (Kotowski et al., 2024; Hundman et al., 2018) and are the natural next place to try the check.

10. Conclusion

In the case that a feature scaler is used in multiple groups, where each group has their own scale, then a deep model can become into a per group constant, and a tree on the same arrays will return that the solution has been found. None of the pieces here are exotic. One of the default decisions of a standard pipeline is the preprocessing. The model used is just a plain Transformer. The usual one, that the people would pick up is the baseline. The standard defence, the checking of a simple baseline, is constructed so as not to be able to see the defect, which makes it impossible to fail the broken model.

We don't claim the fix, normalisation per group, to be new. The problem to be solved is the detection problem: there is a check which is executed regularly and which certifies a broken configuration, and then another check which is not expensive, but which catches the broken configuration. If the range of the predictions in each group had been given in addition to the summary numbers, we would have spotted ours quickly. On a second dataset, where pooling shrank the signal and the model survived, the same check raised no alarm.

11. Code and Data Availability

All the scripts for the experiments, the JSON artifacts that each number is read from, the code for the figures and the source of this document are available in a public repository (URL withheld for double-anonymized review). A short map of where things are:

`src/ml/` holds the experiments. `norm_ablation.py` produces Table 1, `baseline_invariance.py` Table 2, `diagnostic_runs.py` with `diagnostic_eval.py` Table 4, `second_dataset.py` Table 5 and `prune_economics.py` Table 6. The turbofan data are public, from the NASA Prognostics Center of Excellence repository, and `second_dataset.py` reads them from a folder given on the command line. All of them share the same definition of the train/validation/test partition, in `splits.py`.

`src/ml/model.py` is the Transformer, and `planet_config.py` holds the target list, the cadences and the recorded reason the Moon is excluded.

`reports/` holds the measured artifacts. Every table and figure in this document comes from one of those JSON files, and `multiseed_paper/` holds the runs at each seed behind Table 3.

`docs/RESEARCH_LEDGER.md` is the complete decision history, including two analyses that were invalidated by a positional join and the corrections that replaced them. `docs/LIMITATIONS.md` states what the results do not support.

The JSON creates the tables and figures in this manuscript and not the other way round. No number was typed into the manuscript by hand.

The experiments read per target `.npz` files of roughly 70 MB each, which are derived from a 71 GB mission table that is too large to store in the repository. The scripts below regenerate the artifacts from those extracts. `ORBITGUARD_DATA` points at the full table, and the economics analysis is the only experiment that needs it.

```
export ORBITGUARD_DATA=/path/to/dataset

# Primary results, seed 42
python -m src.ml.norm_ablation          # Table 1
python -m src.ml.baseline_invariance    # Table 2
python -m src.ml.diagnostic_runs        # Table 4, Fig. 6
python -m src.ml.diagnostic_eval
python -m src.ml.second_dataset --data /path/to/CMAPSSData   # Table 5
python -m src.ml.prune_economics        # Table 6

# Replication, one pass per seed (Table 3, Section 5)
for S in 0 1 2 3; do
  python -m src.ml.norm_ablation --seed $S \
      --out reports/multiseed_paper/normalisation_ablation_seed$S.json
  python -m src.ml.baseline_invariance --seed $S \
      --out reports/multiseed_paper/baseline_invariance_seed$S.json
done

# The document
python docs/paper/build/collect_traces.py   # Fig. 2 data
python docs/paper/build/figures.py          # Figs. 1 to 5
python eaai/new_results.py                  # Fig. 6
python docs/paper/build/render_latex.py     # this document
```

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) in order to write and debug code, draft parts of the text, and check references. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

Acciarini, G., Pinto, F., Letizia, F., Martinez-Heras, J.A., Merz, K., Bridges, C., Baydin, A.G., 2021. Kessler: A machine learning library for spacecraft collision avoidance, in: 8th European Conference on Space Debris.

Ba, J.L., Kiros, J.R., Hinton, G.E., 2016. Layer normalization. arXiv preprint arXiv:1607.06450 doi:10.48550/arXiv.1607.06450.

Chaudhary, D.K., Srivastava, S., Kumar, V., 2018. A review on hidden debts in machine learning systems, in: Second International Conference on Green Computing and Internet of Things (ICGCIoT). doi:10.1109/ICGCIOT.2018.8753081.

Chen, T., Guestrin, C., 2016. XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794. doi:10.1145/2939672.2939785.

Deng, J., Chen, X., Jiang, R., Song, X., Tsang, I.W., 2021. ST-Norm: Spatial and temporal normalization for multi-variate time series forecasting, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, pp. 269–278. doi:10.1145/3447548.3467330.

Geirhos, R., Jacobsen, J.H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., Wichmann, F.A., 2020. Shortcut learning in deep neural networks. Nature Machine Intelligence 2, 665–673. doi:10.1038/s42256-020-00257-z.

Grinsztajn, L., Oyallon, E., Varoquaux, G., 2022. Why do tree-based models still outperform deep learning on tabular data?, in: Advances in Neural Information Processing Systems (NeurIPS), pp. 507–520. doi:10.48550/arXiv.2207.08815.

Huang, L., Qin, J., Zhou, Y., Zhu, F., Liu, L., Shao, L., 2023. Normalization techniques in training DNNs: Methodology, analysis and application. IEEE Transactions on Pattern Analysis and Machine Intelligence 45, 10173–10196. doi:10.1109/TPAMI.2023.3250241.

Hughes, S.P., Qureshi, R.H., Cooley, D.S., Parker, J.J.K., 2014. Verification and validation of the general mission analysis tool (GMAT), in: AIAA/AAS Astrodynamics Specialist Conference. doi:10.2514/6.2014-4151.

Hundman, K., Constantinou, V., Laporte, C., Colwell, I., Söderström, T., 2018. Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 387–395. doi:10.1145/3219819.3219845.

Ibrahim, S.K., Ahmed, A., Zeidan, M.A.E., Ziedan, I.E., 2019. Machine learning methods for spacecraft telemetry mining. IEEE Transactions on Aerospace and Electronic Systems 55, 1816–1827. doi:10.1109/TAES.2018.2876586.

Ioffe, S., Szegedy, C., 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: International Conference on Machine Learning (ICML), pp. 448–456. doi:10.48550/arXiv.1502.03167.

Izzo, D., Blazquez, E., Ferede, R., Origer, S., De Wagter, C., de Croon, G.C.H.E., 2024. Optimality principles in spacecraft neural guidance and control. Science Robotics 9, eadi6421. doi:10.1126/scirobotics.adi6421.

Izzo, D., Märtens, M., Pan, B., 2019a. A survey on artificial intelligence trends in spacecraft guidance dynamics and control. Astrodynamics 3, 287–299. doi:10.1007/s42064-018-0053-6.

Izzo, D., Sprague, C., Tailor, D., 2019b. Machine learning and evolutionary techniques in interplanetary trajectory design, in: Optimization in Space Engineering. doi:10.1007/978-3-030-10501-3_8.

Jia, B., Xin, M., 2022. Data-driven surrogate model and admissible-region-based orbital uncertainty propagation. Journal of Aerospace Information Systems 19. doi:10.2514/1.I011107.

Jiang, S., Jiang, Y., Wang, Y., Zhang, X., Zhang, Z., 2023. Anomaly detection in spacecraft telemetry data based on transformer-LSTM, in: International Conference on Intelligent Communication and Networking (ICN). doi:10.1109/ICN60549.2023.10426290.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y., 2017. LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems (NeurIPS), pp. 3146–3154.

Kotowski, K., Haskamp, C., Andrzejewski, J., Ruszczak, B., Nalepa, J., Lakey, D., Collins, P., Kolmas, A., Bartesaghi, M., Martínez-Heras, J., De Canio, G., 2024. European space agency benchmark for anomaly detection in satellite telemetry. arXiv preprint arXiv:2406.17826 doi:10.48550/arXiv.2406.17826.

Lapuschkin, S., Wäldchen, S., Binder, A., Montavon, G., Samek, W., Müller, K., 2019. Unmasking clever hans predictors and assessing what ma-

chines really learn. Nature Communications 10, 1096. doi:10.1038/s41467-019-08987-4.

Li, B., Huang, J., Feng, Y., Wang, F., Sang, J., 2020a. A machine learning-based approach for improved orbit predictions of LEO space debris with sparse tracking data from a single station. IEEE Transactions on Aerospace and Electronic Systems 56, 4253–4268. doi:10.1109/TAES.2020.2989067.

Li, H., Chen, S., Izzo, D., Baoyin, H., 2020b. Deep networks as approximators of optimal low-thrust and multi-impulse cost in multitarget missions. Acta Astronautica 166, 469–481. doi:10.1016/j.actaastro.2019.09.023.

Li, X., Ding, Q., Sun, J.Q., 2018. Remaining useful life estimation in prognostics using deep convolution neural networks. Reliability Engineering & System Safety 172, 1–11. doi:10.1016/j.ress.2017.11.021.

Massari, M., Di Lizia, P., Rasotto, M., 2017. Nonlinear uncertainty propagation in astrodynamics using differential algebra and graphics processing units. Journal of Aerospace Information Systems 14, 493–503. doi:10.2514/1.I010535.

Naik, K.R., Holmgren, A., Kenworthy, J., 2020. Using machine learning to automatically detect anomalous spacecraft behavior from telemetry data, in: IEEE Aerospace Conference. doi:10.1109/AERO47225.2020.9172726.

Nickel, C., Dichmann, D.J., Lebois, R., Lutz, S., Parker, J.J.K., 2016. Monte carlo analysis as a trajectory design driver for the TESS mission, in: AIAA/AAS Astrodynamics Specialist Conference. doi:10.2514/6.2016-5665.

Pasaribu, J., Yudistira, N., Mahmudy, W.F., 2024. Tabular data classification and regression: XGBoost or deep learning with retrieval-augmented generation. IEEE Access 12, 191719–191732. doi:10.1109/ACCESS.2024.3518205.

Passalis, N., Tefas, A., Kanniainen, J., Gabbouj, M., Iosifidis, A., 2020. Deep adaptive input normalization for time series forecasting. IEEE Transactions on Neural Networks and Learning Systems 31, 3760–3765. doi:10.1109/TNNLS.2019.2944933.

Sánchez-Sánchez, C., Izzo, D., 2018. Real-time optimal control via deep neural networks: Study on landing problems. Journal of Guidance, Control, and Dynamics 41, 1122–1135. doi:10.2514/1.G002357.

Saxena, A., Goebel, K., Simon, D., Eklund, N., 2008. Damage propagation modeling for aircraft engine run-to-failure simulation, in: International Conference on Prognostics and Health Management, pp. 1–9. doi:10.1109/PHM.2008.4711414.

Shah, H., Tamuly, K., Raghunathan, A., Jain, P., Netrapalli, P., 2020. The pitfalls of simplicity bias in neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), pp. 9573–9585.

Shwartz-Ziv, R., Armon, A., 2022. Tabular data: Deep learning is not all you need. Information Fusion 81, 84–90. doi:10.1016/j.inffus.2021.11.011.

Sola, J., Sevilla, J., 1997. Importance of input data normalization for the application of neural networks to complex industrial problems. IEEE Transactions on Nuclear Science 44, 1464–1468. doi:10.1109/23.589532.

Stevenson, E., Rodríguez-Fernández, V., Urrutxua, H., Camacho, D., 2023. Benchmarking deep learning approaches for all-vs-all conjunction screening. Advances in Space Research 72, 2660–2675. doi:10.1016/j.asr.2023.01.036.

Ulyanov, D., Vedaldi, A., Lempitsky, V., 2016. Instance normalization: The missing ingredient for fast stylization. arXiv preprint arXiv:1607.08022 doi:10.48550/arXiv.1607.08022.

Uriot, T., Izzo, D., Simões, L.F., Abay, R., Einecke, N., Rebhan, S., Martinez-Heras, J., Letizia, F., Siminski, J., Merz, K., 2020. Spacecraft collision avoidance challenge: design and results of a machine learning competition. arXiv preprint arXiv:2008.03069 doi:10.48550/arXiv.2008.03069.

Wu, Y., He, K., 2018. Group normalization, in: Proceedings of the European Conference on Computer Vision (ECCV), pp. 3–19. doi:10.48550/arXiv.1803.08494.

Yin, J., Wang, F., Shankar, M., 2022. Strategies for integrating deep learning surrogate models with HPC simulation applications, in: IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). doi:10.1109/IPDPSW55747.2022.00222.

Zavoli, A., Federici, L., 2021. Reinforcement learning for robust trajectory design of interplanetary missions. Journal of Guidance, Control, and Dynamics 44, 1440–1453. doi:10.2514/1.G005794.

Zhou, X., Qiao, D., Li, X., 2024. Neural network-based method for orbit uncertainty propagation and estimation. IEEE Transactions on Aerospace and Electronic Systems 60, 1176–1193. doi:10.1109/TAES.2023.3332566.